Unitree G1 Air — WebRTC Control: Findings & Capabilities

Reverse-engineered guide to controlling a **Unitree G1 Air** from a PC over WebRTC (no EDU, no on-board ROS2, no DDS). Built by decompiling the Unitree Explore app and probing the robot. Updated June 2026.

---

0. What you can do RIGHT NOW (proven)

Capability	Status	How
Connect from Python		`unitree_webrtc_connect` LocalAP, robot AP `192.168.12.1`
SLAM: start/stop/save map		`rt/api/slam_operate/request` (api 1801/1802/1803)
Receive mapping odom / info		`rt/unitree/slam_mapping/odom`, `rt/slam_info`
Teleop: walk / turn / strafe		`rt/wirelesscontroller` {lx,ly,rx,ry} @20Hz (robot in walk mode)
Play built-in arm gestures		`rt/api/arm/request` api 7106
Create/replay custom gestures (Demo Teaching)		record in app or api 7110/7113; play api 7108
Read arm joint angles		`rt/lf/lowstate` motor_state[i].q
Telemetry (IMU, joints, battery, FSM)		`rt/lf/lowstate`, `rt/lf/sportmodestate`, `rt/slam_info`
List robot services		`rt/api/robot_state/request` api 1003 / `rt/servicestate`
Camera stream		video channel (RealSense/front cam)
LIVE LiDAR point cloud + SLAM	*	*not over plain WebRTC — captured from the app's WebView over USB (see §10)

What does NOT work on the Air (limits)

- **LiDAR point cloud over plain WebRTC** — the robot only sends `odom` + `slam_info` to a third-party WebRTC client; the cloud is rendered only inside the app's WebView. Workaround: capture it from the WebView over USB — see **§10** (this DOES give live laser).
- **Hold a custom arm pose while walking** — needs low-level arm SDK (`rt/arm_sdk`) which the Air forwards but does NOT actuate (EDU-only capability). Gestures/teaching DO move the arm but **lock the legs** by design.
- **DDS / CycloneDDS** — internal bus isolated; not exposed on the WiFi AP (EDU-only).
- **bashrunner (remote scripts)** — service not running / no response over WebRTC.
- **Go2 sport API** (`rt/api/sport/request` Move/BalanceStand) — returns 3203 (it's the dog API).

---

1. The platform reality

- G1 **Air** = base tier: no Jetson, no developer Ethernet, no SSH. Only access = WiFi AP (`192.168.12.0/24`, robot at `192.168.12.1`) + **WebRTC** signaling on `:8081/offer` (legacy, no AES key on this firmware).
- Library: community `legion1581/unitree_webrtc_connect` (LocalAP mode).
- **One WebRTC session at a time** — close the phone app before running scripts.

---

2. SLAM API (rt/api/slam_operate/request)

Request envelope: `publish_request_new("rt/api/slam_operate/request", {"api_id":<id>, "parameter":{"data":<dat`

Operation	api_id	data
Start mapping	1801	`{"slam_type":"indoor"}`
End + save	1802	`{"address":"/unitree/data/unitree_slam/<name>.pcd"}`
Cancel	1803	—
Start relocalization	1804	`{x,y,z,q_x,q_y,q_z,q_w, address}`
Close relocalization	1805	—
Navigate to point	1102	`{"targetPose":{x,y,z,q_x..q_w}, "mode":1}`
Pause/resume nav	1201/1202	—
File read/write (map)	1934/1933	`{address, ...}` (chunked)

Topics received: `rt/unitree/slam_mapping/points` (cloud, while driving), `.../odom`, `rt/slam_info`.

3. Teleop / walking (rt/wirelesscontroller)

Publish `{lx,ly,rx,ry}` (floats, no "keys") at **20 Hz** continuously; zeros to stop. Robot must be in walk mode (standing, as when driven by the remote). Mapping: ly=fwd/back, lx=strafe, rx=yaw, ry=euler.

4. Arm: gestures & Demo Teaching (rt/api/arm/request)

`publish_request_new("rt/api/arm/request", {"api_id":<id>, "parameter":{...}})`.

Operation	api_id	param
Built-in upper-limb action	7106 (G1UpperLimbs)	`{"data":<action_id>}`
List learned actions	7107	`{}`
Play learned action	7108	`{"action_name":"<name>"}`
Start recording	7110	`{"action_name":"<name>"}`
Stop record/play	7113	—
Pause / delete / rename	7111 / 7112 / 7109	—
Release arm (fsm 550)	7100	`{fsm_id:550,api_id:2,motion_paused:false}` → rejected 3103 on Air

Note: playing any arm action enters an action FSM that **stops the legs**.

5. Low-level arm (rt/arm_sdk) — read-only on the Air

Format (unitree_sdk2, msg unitree_hg LowCmd): `motor_cmd[i]={mode,q,dq,tau,kp,kd}`, right arm = indices 22–28, weight at `motor_cmd[29].q`. On EDU this holds an arm pose while walking (weight blend). On the **Air the bridge forwards it but no controller actuates it** → arm doesn't move (the "stiffness" felt is the robot's normal standing posture). Reading current pose works via `rt/lf/lowstate`.

6. Other reachable APIs (DogApiId)

motion_switcher (GET 1001 / SWITCH 1002 / RELEASE 1003 / silent 1004-1005), config, voice, audiohub, robot_state, action_store (UniStore: list 1001 / run 1005 / stop 1006), basic_service.

7. Scripts in this toolkit

Script	Purpose
`slam_g1_mapping.py`	Start/stop/save SLAM + receive odom

Script	Purpose
`g1_teleop.py`	Walk/turn/strafe (+ armwalk/walkthenplay tests)
`g1_arm_teaching.py`	List/play/record arm gestures
`g1_arm_sdk.py`	Read arm joints; test low-level arm_sdk
`g1_coffee_walk.py` / `g1_arm_during_walk.py`	(experimental) arm pose + walk attempts
`g1_slam_viz.py`	Live robot trajectory/pose viz (from odom, pure WebRTC)
`g1_inspector_bridge.py`	**LIVE LiDAR cloud + robot path, via WebView over USB (§10)**
`g1_cloud_ws_viz.py`	Live cloud viz over a WebSocket (if no AP isolation)
`g1_map_viz.py`	View an exported `.json` point-cloud map (2D/3D)
`g1_slaminfo_dump.py` / `g1_slam_capture_points.py` / `g1_slam_sniff.py`	SLAM data diagnostics
`g1_bashrunner.py`	Probe the (closed) script runner
`dump_services.py`, `query_api.py`, `discover_slam_api.py`	Diagnostics

8. How it was discovered

G1 Air app (`com.unitree.b2dog`) is Baidu-hardened; unpacked with **frida-dexdump** on an arm64 Android emulator. SLAM is a WebView — launched headless via `adb shell am start -n com.unitree.b2dog/com.unitree.godog.ui.acti` and inspected with **chrome://inspect**, revealing the real topics/api_ids. Arm/teaching APIs from the decompiled Kotlin (TeachPlayViewModel, DogApiId).

9. To get "hold a custom arm pose WHILE walking"

Needs a **G1 EDU** (or wired access): the arm SDK over DDS (`rt/arm_sdk` + weight) does exactly this and is well documented in `unitree_sdk2` (`g1_arm7_sdk_dds_example`). Not available on the Air.

---

10. LIVE LiDAR point cloud + SLAM in Python (WebView bridge over USB)

The robot does **not** stream the point cloud to a third-party WebRTC client (only `odom` + `slam_info`). But the **Unitree Explore app's SLAM screen is a WebView** that *does* receive and decode the cloud (Three.js). We tap that WebView **over the USB cable** (no WiFi → immune to the robot AP isolating its clients) and forward the decoded points to a Python live viewer.

Why USB (not WiFi)

The robot's WiFi AP **isolates clients**, so a WebSocket from the iPhone WebView to a Python server on the Mac (both on the robot AP) just hangs. The Safari Web Inspector / `ios-webkit-debug-proxy` channel runs over the **USB cable** (usbmuxd), bypassing that entirely.

One-time setup

1. **iPhone:** Settings → Safari → Advanced → **Web Inspector: ON**.
2. **Mac Safari (optional, for manual inspection):** Safari → Settings → Advanced → **"Show features for web developers"** → a **Develop** menu appears.
3. **Connect the iPhone to the Mac with the USB cable** and tap **Trust**.
4. Install the proxy: `brew install ios-webkit-debug-proxy`.
5. Python deps: `pip install websocket-client requests matplotlib numpy`.

Run it

1. On the **iPhone**: open Unitree Explore, connect to the robot, go to **Navigator / SLAM** (the screen where the 3D map renders). Leave it there.

2. **Close any Safari Web Inspector window** for that page — only **one debugger per page**, and we want it to be Python.
3. **Terminal 1** — start the proxy (leave running):

   ```
   ios_webkit_debug_proxy
   ```

4. (Optional check) discover the device + page:

   ```
   curl -s http://localhost:9221/json        # device -> "url":"localhost:9222"
   curl -s http://localhost:9222/json        # pages  -> title "B2App", url .../#/newSlam
   ```

5. **Terminal 2** — run the bridge/viewer:

   ```
   cd ~/unitree_webrtc_connect && source .venv/bin/activate
   python ".../g1_inspector_bridge.py"        # 2D top-down (default)
   python ".../g1_inspector_bridge.py" 3d      # rotatable 3D
   ```

6. **Drive the robot** with the remote → the LiDAR cloud builds live, with the **red ball** = current robot pose and the **red line** = path travelled.

How the bridge works

- `ios-webkit-debug-proxy` exposes the WebView's inspector (a WebKit/CDP protocol) on `localhost:9221/9222`. Modern iOS wraps commands in `Target.sendMessageToTarget` / `Target.dispatchMessageFromTarget` — the bridge handles that.
- It injects a JS hook into the page via `Runtime.evaluate`:
  - **Cloud:** wraps `Worker.prototype.postMessage` to capture the worker's **decoded** output (`{type:"newMap", data:{ directCount, directOutput:[x,y,z,...] }}`) into `window.__buf`.
  - **Odometry:** wraps `JSON.parse` to catch `rt/unitree/slam_mapping/odom` messages and store `window.__odom = [x,y,z, qx,qy,qz,qw]`.
- Every ~0.4 s it evaluates a poll that reads+clears `window.__buf` and reads `window.__odom`, then accumulates points (5 cm voxel grid) and renders with matplotlib (2D or 3D), drawing the robot pose + path.

Cloud format (for reference)

Raw message to the worker: `{header:{frame_id:"map"}, is_dense, xmin..xmax, ymin..ymax, zmin..zmax, data:<ArrayBuffer>}` (voxel-encoded). The worker decodes it to a flat XYZ array (`directOutput`, length `3*directCount`) in the **map** frame — that's what we plot.

Snapshot alternative (no proxy)

With the Safari Web Inspector console open on the SLAM page, you can accumulate the map and export it:

```
// paste once: accumulate into window.__map (5cm voxel dedup)
// ... (hook que mete los puntos en window.__map) ...
copy(JSON.stringify(Object.values(window.__map)));   // to clipboard
```

then on the Mac: `pbpaste > ~/g1/map.json` and `python g1_map_viz.py ~/g1/map.json [3d]`. (Note: Safari's `copy()` is unreliable for large data — the live USB bridge is the robust path.)

Safety / legal

Reverse engineering done on your own robot for interoperability; do not redistribute the APK/dex/keys. Robot tests: stand via remote, clear area or gantry, remote as kill-switch, low speeds first.